[Web발신]"너는 NestJS를 잘못 쓰고 있었다." GraphQL + Dynamic Include + Interceptor 신개념 New 빌드로 N+1 문제부터 DB Overfetching까지 해결한 '역대급 프레임워크' 등장...이거 지금 못 막습니다 💀💀

Google Developer Groups
Sungkonghoe University



Younha Lee 이윤하
GDG on Campus SKHU 24-25 Organizer

목차

1. GraphQL의 특징 복습
2. N+1 문제란?
3. Include로 해결해보기
4. 동적으로 Include 만들어보기
5. Interceptor & Execution Context

프로젝트 배경

NestJS + GraphQL + Prisma

GraphQL 복습

```graphql
query GetUserWithPosts($userId: ID!) {
  user(id: $userId) {
    id
    name
    posts {
      title
    }
  }
}
```

```json
{
  "data": {
    "user": {
      "id": "123",
      "name": "김철수",
      "posts": [
        {
          "title": "NestJS로 GraphQL API 만들기"
        },
        {
          "title": "Prisma와 함께하는 데이터베이스 마이그레이션"
        },
        {
          "title": "N+1 문제 해결을 위한 인터셉터 활용법"
        }
      ]
    }
  }
}
```

API를 위한 Query Language

클라이언트가 필요로 하는 정보만 주문서처럼 작성해 서버에 보내는 방식

UnderFetching과 OverFetching을 방지할 수 있음

Prisma란?



Node.js 진영에서 사용하는 ORM

DB 조작을 SQL이 아닌, 자바스크립트로 해결할 수 있게 함.

N+1 문제란?

DB에서 데이터를 가져올 때 발생하는 **비효율적인 쿼리 패턴**

어떤 정보를 가져오기 위해 불필요하게 여러 번 질문을 반복하는 상황

(오늘은 One To Many 관계에서의 문제를 상정)

N+1 문제가 발생한 상황



"여기 있는 만화책 목록 좀 보여주세요!



[진격의 거인, 주술회전, 원피스....]

N+1 문제가 발생한 상황





진격거 작가가 누구죠? → 이사야마 하지메 입니다.

주술회전 작가가 누구죠? → 게게 선생입니다.

원피스 작가가 누구죠? → 오다 센세입니다.

.

.

.

.

N+1 문제란?

최초 1번의 만화책 목록 쿼리 이후

각 만화책(N개)마다 작가 정보를 가져오기 위해 N번의 추가 쿼리 발생

총 **1+N** 번의 쿼리 발생

해결 방법

"만화책 목록 가져올 때 각 책의 작가 정보도 **같이** 담아주세요"

```
const booksWithAuthors = await prisma.book.findMany({
  include: {
    author: true
  },
});
```

Prisma의 relationLoadStrategy

```
const users = await prisma.user.findMany({
  relationLoadStrategy: 'join', // or 'query'
  include: {
    posts: true,
  },
})
```

개발자가 relationLoadStrategy를 명시적으로 설정하지 않으면,

Prisma는 내부적인 최적화 로직에 따라 join 전략과 query 전략 중 더 적합하다고

판단되는 방식을 자동으로 선택하여 N+1 문제를 해결

Prisma의 relationLoadStrategy

When to use which load strategy?

- The `join` strategy (default) will be more effective in most scenarios. On PostgreSQL, it uses a combination of `LATERAL JOINs` and JSON aggregation to reduce redundancy in result sets and delegate the work of transforming the query results into the expected JSON structures on the database server. On MySQL, it uses correlated subqueries to fetch the results with a single query.
- There may be edge cases where `query` could be more performant depending on the characteristics of the dataset and query. We recommend that you profile your database queries to identify these situations.
- Use `query` if you want to save resources on the database server and do heavy-lifting of merging and transforming data in the application server which might be easier to scale.

공식 문서에서는 대부분의 경우 join 방식을 사용하는 것이 권장

Include의 문제점 (in GraphQL)

GraphQL의 핵심

"클라이언트가 필요한 데이터만 정확히 요청하고 받을 수 있다"

클라이언트가 books { id title }만 요청했다면,

서버는 author 정보를 보내지 않는 것이 이상적임.

Include의 문제점 (in GraphQL)

```
const booksWithAuthors = await prisma.book.findMany({
  include: {
    author: true
  },
});
```

author가 필요하든 말든

일단 모든 책과 그에 해당하는 작가 정보를 함께 조회

해결법

```
async function booksWithAuthors(includeAuthor: boolean) {
  const queryOptions: Prisma.BookFindManyArgs = {};

  if (includeAuthor) {
    queryOptions.include = {
      author: true,
    };
  }

  return prisma.book.findMany(queryOptions);
}
```

클라이언트가 author의 정보를 요청했는지에 따라
동적으로 include 객체를 생성

* BookFindManyArgs는 Prisma가 생성한 findMany 메서드에 인자로 전달될 수 있는 모든 유효한 옵션들을 정의한 TypeScript 타입

목차

1. GraphQL의 특징 복습
2. N+1 문제란?
3. Include로 해결해보기
4. 동적으로 Include 만들어보기

---

5. Interceptor & Execution Context



Execution Context

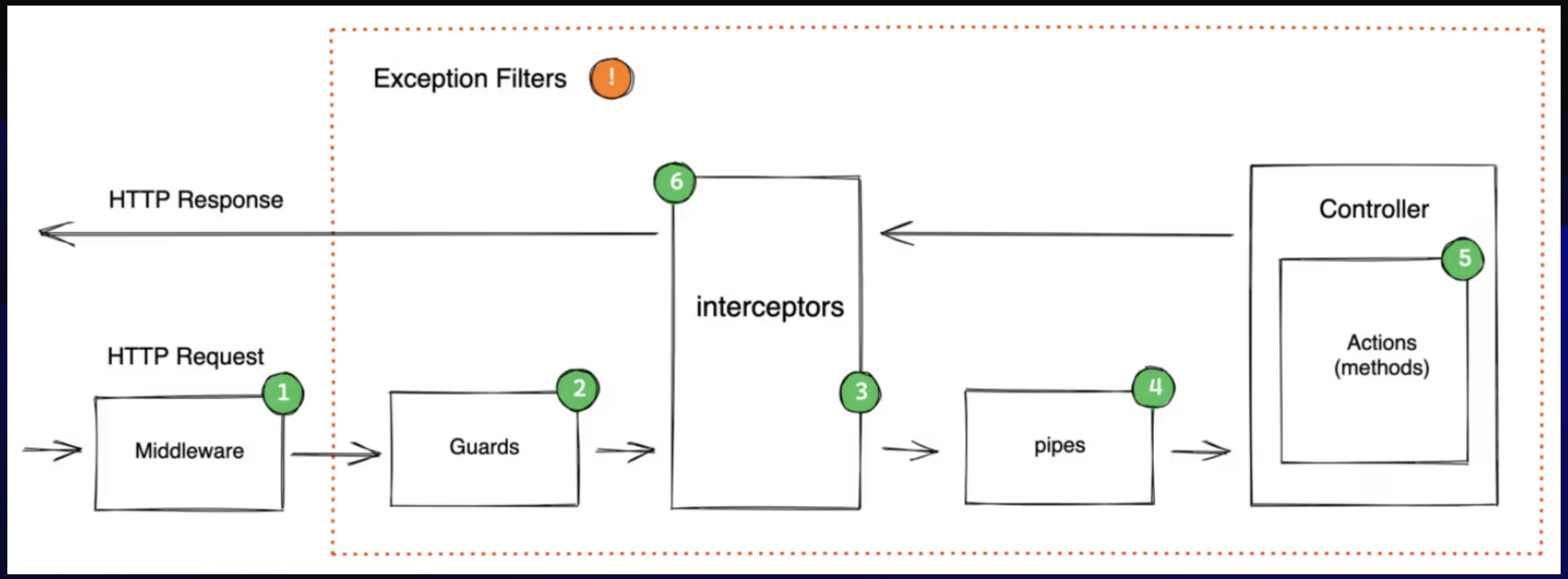
요청에 대한 모든 **상황 정보**와 **메타데이터**를 담고 있는 객체

REST API, GraphQL, WebSocket, RPC 등 다양한 통신 프로토콜 위에서 동작

```
const gqlContext = GqlExecutionContext.create(context);
const info = gqlContext.getInfo<GraphQLResolveInfo>();
```

GraphQL 요청에 특화된 정보로 변환, 해당 context에서 클라이언트가 요청한 **필드**를 가져옴

Interceptor



ResolVer에 도달하기 전에 **GqlExecutionContext** 에 접근해야 함

Request를 가로채 다음 단계로 전달하는 Interceptor가 가장 적합

나머지 구현

1. 유저가 요청한 GraphQL 요청에서 필드를 추출
2. Include 옵션으로 변환
3. Prisma 서비스에 Include를 전달
4. 최적화된 쿼리 실행

요약

1. N+1은 RDB & ORM에서 일어나는 SQL 쿼리 문제
2. Prisma에서는 Include를 사용하여 해결한다.
3. Interceptor에서 GqlExecution Context에 접근하여 요청한 필드에 접근할 수 있다.
4. 추출한 필드로 include를 동적으로 만들어 효율적으로 요청을 보낼 수 있다.